A 1.2.1-2 Representing Data

Table of contents



A 1.2.1 Describe the principal methods of representing data. -A 1.2.2 Explain how binary is used to store data.

This section explores how computers represent data. We'll examine common number systems (binary, hexadecimal, decimal) and conversion methods. Then, we'll dive into how computers use binary to store various data types like integers, text, and multimedia. You'll gain a foundational understanding of how information is encoded and processed digitally.

Key Questions

A1.2.1

- What are the principal methods of representing data in a computer system?
- How can integers be represented and converted between binary, hexadecimal, and decimal?

A1.2.2

- How is binary used to store data, and what are its fundamentals?
- How are various types of data (integers, strings, images, audio, etc.) stored in binary form?

The Foundations of Digital Data

Digital Data can be considered information transformed into discrete, measurable units. This process, called digitization, allows computers to work with various forms of data. The most basic unit is a bit, representing two possible states (0 or 1), like an on/off switch. Bits are typically grouped into bytes (8 bits), forming the foundation for encoding more complex information.

- Computers operate using electrical signals (on/off).
- Binary digits 0 and 1 represent these states (0 = off, 1 = on).
- Complex data representation is built from these two digits.
- This binary system is the foundation of computing.

Representing Numbers

Binary

- Binary (Base-2) is the foundation of computing
- 0 and 1 represent electrical states (off/on)
- All data types (numbers, text, images, sound) are represented in binary
- Known as bits (binary digits)
- Enables universal data representation and processing

Binary Details

A unit of information expressed as either a 0 or 1 in binary notation (A **B**inary dig**IT**). Using or denoting a numerical notation system with 2 as a base.

Eights	Fours	Twos	Units	.	Halves	Quarters	Eights
2^3	2^2	2^1	2^0	.	2^{-1}	2^{-2}	2^{-3}
8	4	2	1	.	0.5	0.25	0.125

It is generally accepted that the term byte refers to 8 bits taken together. Bits in a byte read from right to left and are numbered from 0 to 7.

Hexadecimal

- Hexadecimal: Base-16 number system.
- Uses digits 0-9 and letters A-F.
- It's more compact than binary.
- Each digit represents four bits.
- Easier for humans to read/write large numbers.

Hexadecimal Details

Relating to or denoting a system of numbers and arithmetic based on the number 16. This system uses numbers and letters to represent any number no matter how large or small. The numbers and letters used are:
0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F

Decimal	0	1	2	3	4	5	6	7	8	9	10
Hexadecimal	0	1	2	3	4	5	6	7	8	9	A

Decimal

- Decimal: Base-10 number system (0-9).
- Used in everyday life; familiar to humans.
- Computers convert decimal to binary for processing.
- It is inefficient for computers to store and process directly.
- Conversion is a fundamental step in computing.

Decimal Details

Relating to or denoting a system of numbers and arithmetic based on the number ten, tenth parts, and powers of ten.
This system uses the numbers 0, 1, 2, 3, 4, 5, 6, 7, 8 and 9 to represent any number no matter how large or small.

Thousands	Hundreds	Tens	Unit	.	Tenths	Hundredths	
10^3	10^2	10^1	10^0	·	10^{-1}	10^{-2}	
1000	100	10	1	.	0.1	0.01	

So for example, the number 3285:
(3*1000)+(2*100)+(8*10)+(5*1) = 3285
This is also represented as 3285_{10} to indicate the number base.

Number Conversions

Decimal to Binary

Binary to Decimal

Hexadecimal to binary

Converting from hexadecimal is also fairly easy. Simply convert each hexadecimal digit and write the 4-bit binary code.
Example: Convert $3FD4_{16}$ To binary

Converting each hexadecimal digit to a nibble:

Hexadecimal	3	F	D	4
Binary	0011	1111	1101	0100

$3FD4_{16} = 0011\ 1111\ 1101\ 0100_2$

Binary to hexadecimal

Converting from binary to hexadecimal is also a fairly easy process and is done as follows:

- Starting from the right, split the binary number into groups of 4 bits
- If the last group is less than 4 bits, add 0s to the left.

- Take each group of 4 bits and convert to hexadecimal.

Example: Convert 1100111010011_2 to Hexadecimal

Split into groups of 4. Add 0s if required
0001 1001 1101 0011

Convert each byte to hexadecimal

Binary	0001	1001	1101	0011
Hexadecimal	1	9	D	3

$1100111010011_2 = 19D3_{16}$

Decimal to Hex - Method 1

Example: What is 2004_{10} in hex?

The first step is to draw a table starting from the right with 160 and incrementing the power by one each time to get the following:

256	16	1

Add the number that is closest but not bigger than the number you want.
Subtract this from the original number
Keep repeating this with the remainder.

7*256= 1792

2004-1792 = 212

13*16 = 208 ($13_{10} = D_{16)}$

212-208 = 4

Decimal	256	16	1
Hexadecimal	7	D	4

Decimal to Hex - Method 2

This method involves successive division by 16. The remainders are read from bottom to top to give the binary number. Using an example of conversion of 2004_{10} to hexadecimal we get.

2004_{10}	

2004/16 = 125 remainder 4	4_{16}
125/16 = 7 remainder 13 ($13_{10} = D_{16}$)	D_{16}
7/16 = 0 remainder 7	7_{16}

Read the remainder from bottom to top, and we get $7D4_{16}$

Hexadecimal to decimal

To convert Hexadecimal to decimal is fairly easy:

- Take each hexadecimal digit and multiply it by its decimal place value.
- Then add them together to find the value.

Example: Convert $3A8_{16}$ to decimal

Hexadecimal	3	A	8
Calculation	(3*256)	(10*16)	(8*1)
Decimal	768	160	8

$3A8_{16} = (768+160+8) = 936_{10}$

Storing data as binary

Binary is a base-two number system. A few key terms describe groups of binary data, as shown in the table below.

Name	Binary size
bit	1
Nibble	4
Byte	8
Word	Larger group of bits (16, 32, 64), for processing data.

All data is stored in a computer as binary digits (bits). Other data types include:

- Numbers: Stored directly as binary
- Text: Characters encoded using ASCII or Unicode

- Images: Grid of pixels, each colour as binary
- Audio: Sampled sound waves, amplitude stored as binary

	Definition	Representation	Space	Example
Strings	A sequence of characters (letters, numbers, symbols).	Each character is assigned a unique numerical code (usually Unicode).	The space taken depends on the character encoding and the length of the string.	The string "Hello" m be repres in Hex as ASCII: 72 108 108 1 Unicode (8): 48 65 6f
Character	Individual symbols (letters, numbers, punctuation marks).	Similar to strings, each character is assigned a unique numerical code.	The space taken depends on the character encoding. In JAVA a "char" is 16 bits	The chara 'A' might represent as: ASCII: 65 Unicode (8): 41
Integers	Whole numbers (positive, negative, or zero).	Typically stored in binary format (using only 0s and 1s).	The space taken depends on the size of the integer (e.g., 8-bit, 16-bit, 32-bit).	In JAVA a integer (ir assigned bits and t range is -2147483 to 2147483
Images/colours	Visual attributes are represented by combinations of red, green, and blue (RGB) values.	Each colour component (R, G, B) is typically stored as an 8-bit integer (0-255).	24 bits (3 bytes) are usually needed to represent a single colour.	Hexadeci Represer is a short way to represent colours u base-16 numbers. red is (FF,00,00 With 8 bit compone there are (2^8) poss values for each, lea to a total 256 x 256 256 = 16,777,21 possible colours.

Audio	Audio is represented by capturing sound waves' amplitude (intensity) over time with a certain sample rate frequency.	The amplitude of the sound wave is measured at regular intervals (sampling) and stored as a binary number.	The number of bits used to store each sample (bit depth) determines the accuracy of the representation. Common bit depths are 16 bits (CD quality) and 24 bits (studio quality).	A 16-bit a sample c have 2^{16} (65,536) possible values represent different amplitude levels.

Unicode and ASCII

The Need for a Universal Character Set

ASCII (American Standard Code for Information Interchange), developed in the 1960s, could only represent 128 characters. This was sufficient for English and a few other Western European languages but inadequate for representing characters from other languages and scripts.
Different regions started creating their own character encoding standards to accommodate their languages, leading to fragmentation and compatibility issues.

Birth of Unicode

In the late 1980s, a group of engineers from Xerox and Apple began working on a universal character set that could encompass all the world's languages. This initiative evolved into Unicode, which aimed to assign a unique number (code point) to every character, regardless of the platform, program, or language. In 1991, the Unicode Consortium was formed to maintain and develop the Unicode standard.

Unicode has an expansive character set and currently supports over 143,000 characters, covering a vast range of languages and scripts. It also includes symbols, emojis, and other special characters.
Unicode uses different encoding schemes like UTF-8, UTF-16, and UTF-32 to represent characters.
UTF-8, the most common encoding on the web, uses 1 to 4 bytes per character, providing backward compatibility with ASCII while efficiently representing a wide range of characters.

Audio formats

Different audio formats use various compression techniques to reduce file size:

- **Uncompressed:** WAV
- **Lossy Compression:** MP3, AAC (reduce file size by discarding some audio data)
- **Lossless Compression:** FLAC (reduce file size without losing any audio data)

Compression is covered later in the syllabus.